Университет ИТМО

Факультет программной инженерии и компьютерной техники
Образовательная программа
«Системное и прикладное программное обеспечение»

Курсовая работа
По дисциплине «Информационные системы»

Информационная система для поиска и возврата потерянных вещей «Потеряшки»

Этап 3

Выполнил студент группы [номер группы]
Евграфов Артём Андреевич

Проверил:
[ФИО преподавателя]

Санкт-Петербург 2026

Содержание

Задание третьего этапа..... 2
16. Архитектура реализации..... 2
17. Уровень хранения..... 6
18. Уровень бизнес-логики..... 9
19. Проверка и демонстрация на helios..... 13
20. Результаты третьего этапа..... 16
Источники третьего этапа..... 16

Задание третьего этапа

1. Изобразить диаграмму классов, представляющую общую архитектуру системы.

2. Реализовать уровень хранения на основе базы данных второго этапа.

3. Использовать на уровне хранения функции и процедуры PL/pgSQL второго этапа. Не заменять их аналогичными запросами или алгоритмами приложения.

4. Реализовать уровень бизнес-логики на основе процессов первого этапа и уровня хранения.

5. Составить отчёт.

16. Архитектура реализации

16.1. Состав приложения

Реализация третьего этапа находится в каталоге backend. Приложение представляет собой модульный монолит на Java 17 и Spring Boot 3.5.16. Для доступа к данным используются Spring Data JPA, Hibernate и JdbcTemplate. Lombok формирует конструкторы и методы чтения свойств; Jakarta Validation проверяет входные данные объявления. Пароли обрабатываются BCrypt из Spring Security Crypto. Сборка выполняется Gradle Wrapper 8.14.3.

Основные технологические решения соответствуют проекту MyFilmList: Java, Spring Boot, Hibernate/JPA, PostgreSQL, Lombok и Gradle. Версия Spring Boot обновлена в пределах ветки 3.x. Пользовательский интерфейс на Thymeleaf и транспортный слой Spring MVC относятся к следующему этапу. В третьем этапе сервисы вызываются из Java-кода, интеграционных тестов и демонстрационного ApplicationRunner. Приложение не открывает HTTP-порт.

Приложение разделено на пакеты domain, persistence и service. Классы domain отображают таблицы второго этапа. Репозитории persistence выполняют чтение, обычные операции записи и вызовы функций PostgreSQL. Классы service проверяют полномочия участников и управляют транзакциями бизнес-процессов. PoteryashkiApplication собирает Spring-контекст, а DemoRunner запускает демонстрацию на установленных данных.

Таблица 17 — Основные классы реализации

Класс	Назначение
PoteryashkiApplication	Запуск приложения и конфигурация BCrypt
DemoRunner	Демонстрация входа, поиска, справочника и уведомлений через сервисы
AccountService	Регистрация, подтверждение почты, проверка профиля, вход и восстановление пароля
SubscriptionService	Создание платёжного заказа, учебная оплата и периоды подписки
ListingService	Объявления, модерация, поиск, закрытые признаки, метаданные фотографий и справочник организаций
ReturnService	Заявки на находку, сообщения, резервирование и подтверждение передачи
AuctionService	Правовое основание продажи, создание лота, ставки и завершение торгов
AdministrationService	Жалобы, блокировки, роли, тарифы, справочник, уведомления и аудит
OutboxService	Транзакционная доставка событий во внутренние уведомления
AccessPolicy	Проверка пользователя, подписки и служебных ролей; контекст аудита
DatabaseFunctions	Вызовы существующих функций и процедуры PL/pgSQL
SqlStore	Параметризованный CRUD и явные проекции записей
UserRepository, ListingRepository	Чтение неизменяемых JPA-сущностей
SqlNames, TableNamingStrategy	Общий префикс таблиц и изоляция интеграционных проверок

16.2. Диаграммы классов

Общая диаграмма показывает предметные сервисы, общий контроль доступа и классы хранения. На детальных диаграммах показаны репозитории, сущности и процессы возврата и аукциона. Исходники PlantUML позволяют менять состав классов и зависимостей; обзорное изображение построено непосредственно по этому исходнику.

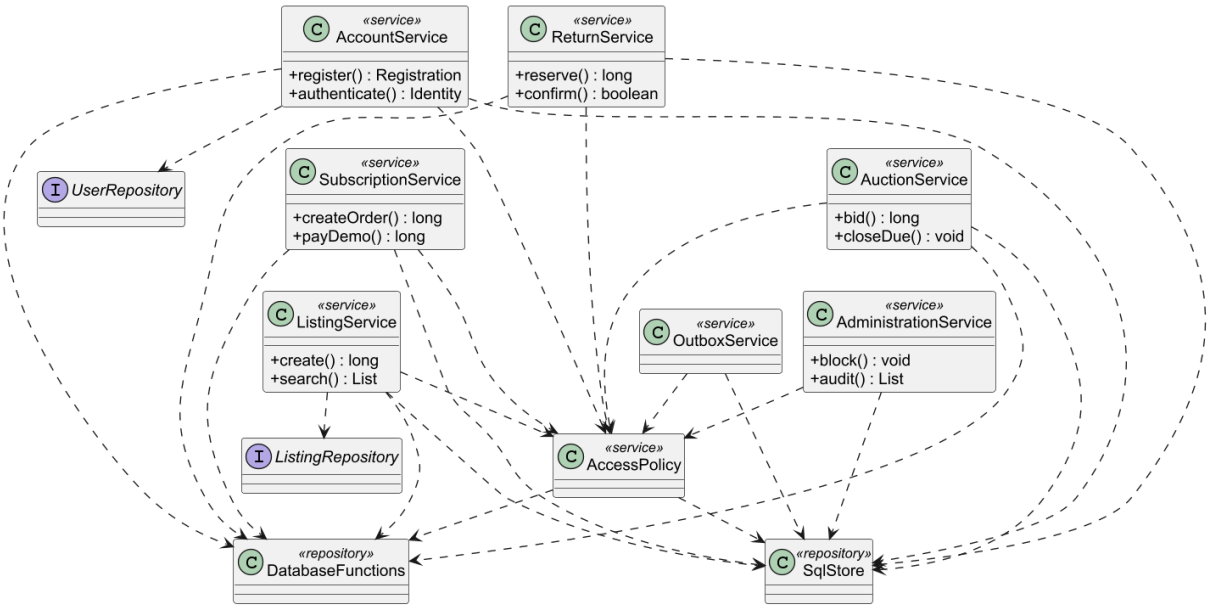


Рисунок 11 — Общая диаграмма классов

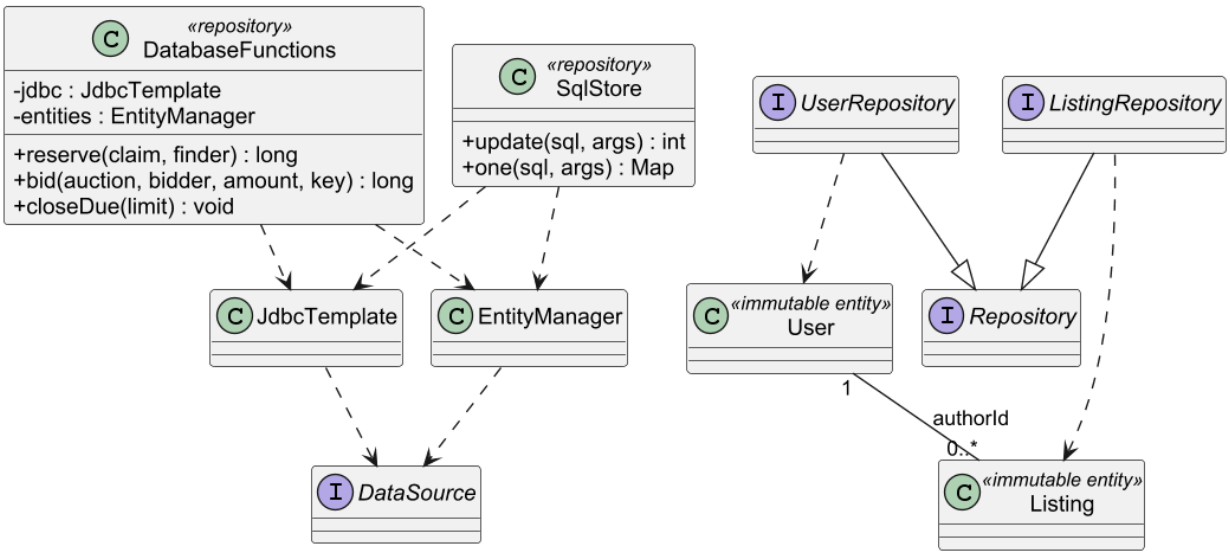


Рисунок 12 — Классы уровня хранения

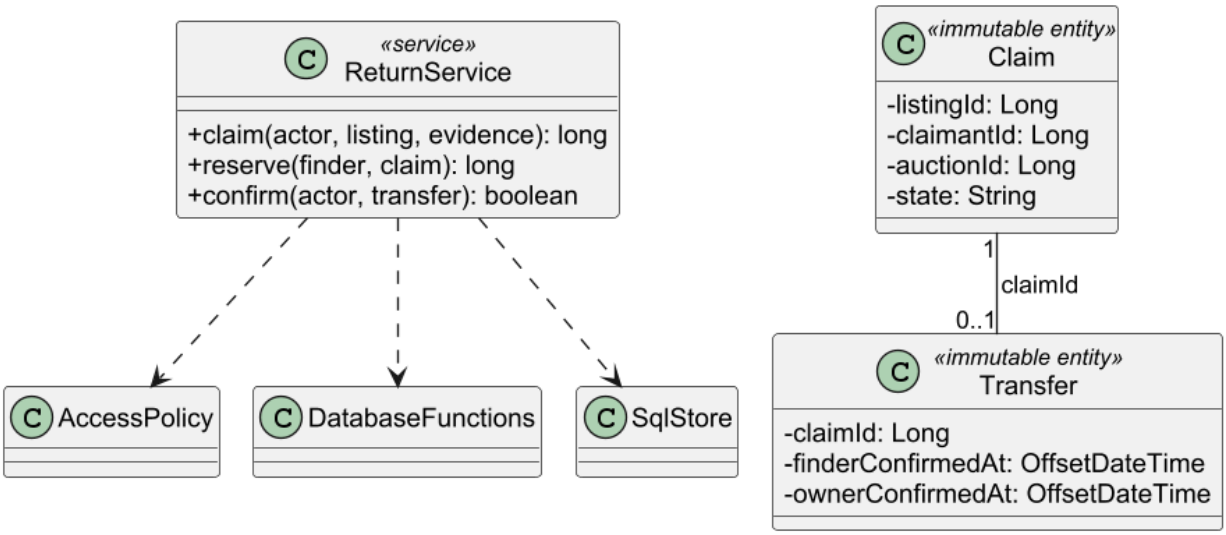


Рисунок 13 — Классы процесса возврата

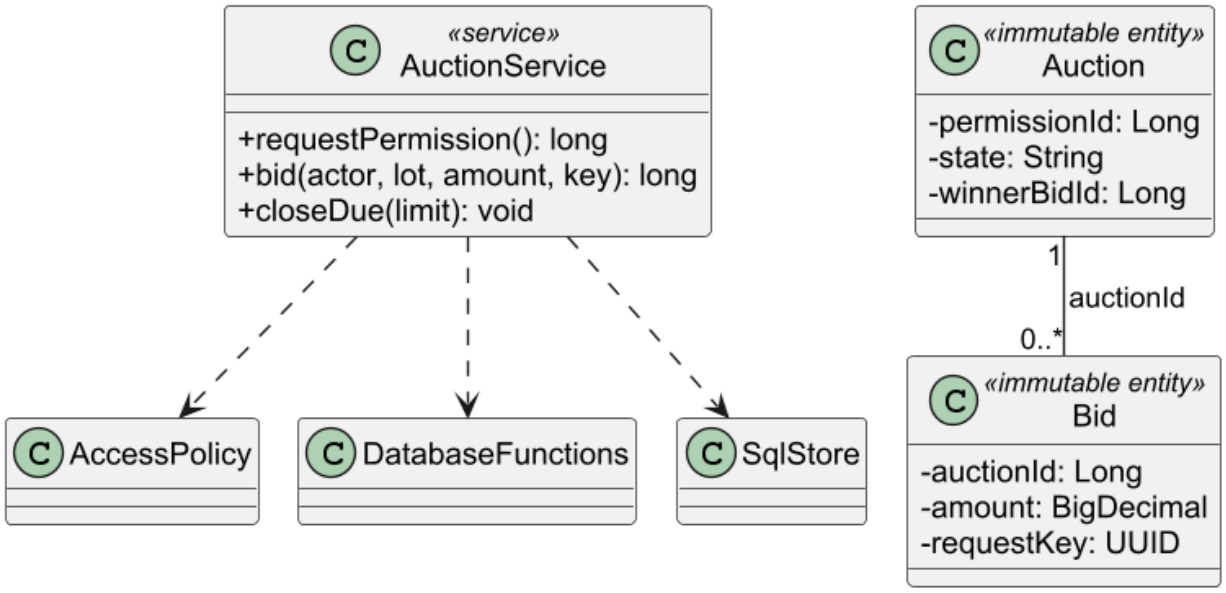


Рисунок 14 — Классы процесса аукциона

Стрелка зависимости означает использование класса. Пустой треугольник в исходниках PlantUML показывает наследование интерфейса Repository. Связи между сущностями обозначают внешние ключи базы данных, а не автоматическую загрузку графа объектов Hibernate. Например, Claim связан с Transfer как 1:0..1, а Auction с Bid как 1:0..*. Составной ключ UserRole сохраняет отношение M:N пользователей и ролей.

17. Уровень хранения

17.1. Отображение базы данных

В пакете domain определены 27 JPA-сущностей, соответствующих всем таблицам второго этапа. UserRole использует составной идентификатор UserRoleKey. Для bigint применяется Long, для integer — Integer, для smallint — Short, для денежных значений numeric — BigDecimal. Моменты времени представлены OffsetDateTime; обмен выполняется в UTC. Для jsonb задан JDBC-тип JSON. Вычисляемый поисковый вектор формируется PostgreSQL и не используется как изменяемое поле JPA.

Все сущности отмечены Hibernate-аннотацией Immutable. UserRepository и ListingRepository предоставляют только операции чтения и не имеют save или delete. Запись выполняется явно через SqlStore и DatabaseFunctions. Это отделяет объектные проекции от переходов состояний и исключает сохранение критичных изменений через обычный JPA-репозиторий.

Таблица 18 — Группы JPA-сущностей

Группа	Классы
Аккаунты и полномочия	User, Role, UserRole, RefreshToken, Verification, VerificationToken
Подписки и платежи	Tariff, PaymentOrder, PaymentEvent, Subscription
Справочники и объявления	Organization, Location, Category, Listing, ListingImage, PrivateAttribute
Возврат вещи	Claim, Conversation, Message, Transfer, Complaint
Аукцион	AuctionPermission, Auction, Bid
События и аудит	Notification, OutboxEvent, AuditEntry

SqlStore использует JdbcTemplate и параметры PreparedStatement. Наименование префикса проверяется по ограниченному шаблону; ввод пользователя не включается в SQL-идентификаторы. На helios применяется префикс lf_, а в проверочном наборе — lf3check_. JPA и JDBC получают одинаковые значения префикса и работают с одним DataSource.

После записи через JDBC контекст EntityManager очищается. Поэтому JPA-чтение в той же транзакции не возвращает устаревшее состояние объекта, изменённого SQL-функцией или триггером. Денежные значения не

переводятся в double и проверяются на положительность, размер и число десятичных знаков.

17.2. Использование PL/pgSQL второго этапа

Для критичных операций DatabaseFunctions вызывает функции PostgreSQL второго этапа. Продление подписки, резервирование, подтверждение передачи, приём ставки и определение победителя выполняются в базе данных в одной транзакции.

Таблица 19 — Соответствие Java-методов операциям PostgreSQL

Метод DatabaseFunctions	**Операция PostgreSQL**	**Где используется**
assertUser	lf_assert_user	AccessPolicy: подтверждение, состояние и подписка
assertModerator	lf_assert_moderator	AccessPolicy: служебные операции
consumeToken	lf_consume_token	AccountService: подтверждение почты и восстановление
activateSubscription	lf_activate_subscription	SubscriptionService: учебная оплата
reserve	lf_reserve_found	ReturnService: согласование возврата
confirm	lf_confirm_transfer	ReturnService: подтверждение передачи
resolveReturn	lf_resolve_return	ReturnService: решение модератора
bid	lf_place_bid	AuctionService: ставка и повтор запроса
finalizeAuction	lf_finalize_auction	AuctionService: завершение конкретного лота
closeDue	lf_close_due_auctions	AuctionService: пакетное завершение торгов
search	lf_search_listings	ListingService: поиск и фильтрация
searchByState	lf_search_listings_state	ListingService: поиск с выбранным публичным статусом

Вызов резервирования содержит только обращение к функции:

```
public long reserve(long claim, long finder) {
return number("select lf_reserve_found(?,?)", claim, finder);
}
```

Ставка также выполняется через функцию:

```
public long bid(long auction, long bidder,
BigDecimal amount, UUID key) {
return number("select lf_place_bid(?,?,?,?)",
auction, bidder, amount, key);
}
```

Процедура завершения вызывается командой CALL. Предел пакета проверяет сама процедура. На уровне сервисов реализованы полномочия инициатора и типы входных значений; расчёт победителя и изменения связанных записей остаются внутри PostgreSQL.

17.3. Транзакции и целостность

Публичные методы сервисов выполняются с Transactional. JpaTransactionManager объединяет JPA и JDBC в одной транзакции. Вызов функции, изменение обычных записей, события outbox и аудит завершаются совместно. Ошибка ограничения или функции вызывает откат. Deferred-ограничения второго этапа проверяются при завершении транзакции; в положительных интеграционных сценариях дополнительно выполняется SET CONSTRAINTS ALL IMMEDIATE.

Автор действия передаётся командой set_config с третьим параметром true. Значение lf.actor_id существует только внутри текущей транзакции и не переносится другому пользователю через пул соединений. Для фоновой задачи автор отсутствует. Триггеры второго этапа сохраняют изменения в неизменяемом журнале.

Для резервирования и ставок сохраняется порядок блокировок SQL-функций: объявление, затем связанная заявка или аукцион. При оплате сначала блокируется пользователь, после чего проверяются его состояние и заказ. Это исключает взаимную блокировку двух подтверждений оплаты при попытке повысить общую блокировку пользователя до исключительной.

Повтор принятой ставки дополнительно проходит проверку актуальной подписки и состояния пользователя. Поэтому заблокированный участник не получает успешный результат повторного запроса только из-за ранее сохранённого ключа. Ключи заявок на оплату и ставок представлены UUID; повтор с другим содержимым отклоняется.

18. Уровень бизнес-логики

18.1. Учётные записи и подписка

При регистрации AccountService нормализует адрес, проверяет имя и пароль, сохраняет BCrypt-хеш и роль user. Одновременно создаются заявка на проверку профиля и одноразовый токен подтверждения почты. В базе хранится SHA-256-хеш токена. Исходное значение является внутренним результатом сервиса для почтового адаптера и не публикуется в каталоге или журнале.

После подтверждения почты профиль рассматривает независимый модератор. Одного подтверждения адреса недостаточно для платных операций: требуется состояние verified. После отказа пользователь может повторно подать профиль, если новая заявка ещё не ожидает рассмотрения.

authenticate проверяет BCrypt и возвращает Identity с идентификатором пользователя и версией сессии. checkIdentity повторно проверяет блокировку и token_version. При смене пароля версия увеличивается триггером, а refresh-токены отзываются в той же транзакции. Подписание access JWT, работа Cookie и HTTP-фильтр будут реализованы на транспортном этапе; Identity используется здесь как внутренний результат проверки учётных данных.

SubscriptionService фиксирует параметры выбранного тарифа в платёжном заказе. Повтор createOrder с тем же ключом возвращает прежний заказ только для того же пользователя и тарифа. Учебный payDemo проверяет владельца и берёт сумму из сохранённого заказа. Он не принимает произвольную сумму от пользователя и не является открытым callback внешнего провайдера. Продление выполняет lf_activate_subscription: сохраняет событие оплаты один раз и добавляет срок после уже оплаченного периода.

18.2. Объявления и справочник метрополитена

ListingService создаёт объявления только для проверенного подписчика. Время и место представлены идентификаторами справочников и моментами события; проверяются тип объявления, заголовок и описание. На пользователя допускается не более десяти новых объявлений за календарные сутки по времени PostgreSQL. Пользователь отправляет черновик на проверку, а независимый модератор публикует или отклоняет его.

Существенное редактирование возвращает объявление в состояние pending посредством триггера. Нельзя менять тип и автора, редактировать вещь во время незавершённого аукциона или после согласования передачи. Закрытые признаки доступны только автору. Публичная карточка строится по явной проекции lf_public_listings и не содержит email, хешей, доказательств заявителей или закрытых признаков.

attachImage сохраняет метаданные уже проверенного объекта: принадлежность ключа объявлению, JPEG/PNG/WebP, размер до 5 МиБ и позицию от 1 до 5. Установка MinIO, загрузка двоичных данных, проверка содержимого изображения и удаление EXIF относятся к файловому адаптеру последующего этапа. Наличие строки метаданных не означает, что двоичный файл уже загружен.

Поиск выполняется через lf_search_listings с текстом, видом, категорией, местом, временем и курсором. Метод searchByState вызывает lf_search_listings_state для отбора по публичному статусу; черновики и скрытые объявления эта функция не возвращает. Справочник организаций содержит официальный источник, дату проверки и признак устаревания после 30 суток. Для Петербургского метрополитена возвращаются телефон и инструкция, подготовленные на втором этапе. Если вещь хранит организация, ReturnService направляет владельца в эту организацию и не создаёт обычный процесс выдачи пользователем.

18.3. Заявки, сообщения и возврат

Заявитель указывает закрытые доказательства владения. Создание заявки и диалога выполняется в одной транзакции. Доступ к сообщениям проверяется как при чтении, так и при записи: участниками являются автор находки и заявитель. Модератор получает доступ при открытой жалобе на эту находку и теряет его после закрытия дела. Ограничение отправки составляет 30 сообщений за минуту. При истёкшей подписке подтверждённый пользователь сохраняет доступ к уже созданному диалогу.

Согласование возврата вызывает lf_reserve_found. Функция проверяет автора находки, незавершённые торги и доступность вещи, принимает одну заявку, резервирует объявление и создаёт передачу. Отмена принятой обычной

заявки согласованно снимает резерв, пока ни одна сторона не подтвердила передачу. Конкурирующая заявка не может получить вторую принятую передачу той же вещи.

Для завершения вызывается lf_confirm_transfer отдельно от каждой стороны. Первое подтверждение сохраняется, но не завершает возврат. Второе переводит заявку, вещь и передачу в завершённое состояние. Уже согласованный возврат можно закончить после истечения подписки. Спор разрешается через lf_resolve_return с полномочиями независимого модератора и обязательным обоснованием.

18.4. Допуск к аукциону и торги

AuctionService принимает заявку на продажу с основанием и ключом документа. Проверку выполняет модератор. Ограничения второго этапа запрещают продажу документов, банковских карт и вещей организации-хранителя. Для основания six_months требуется истечение шести месяцев с официального сообщения о находке, а не с публикации объявления. Самостоятельного автоматического признания права собственности в приложении нет.

После одобрения допуска продавец задаёт начальную цену, шаг и окно торгов. Ставки передаются только в lf_place_bid. PostgreSQL проверяет статус торгов, серверное время, подписку, продавца, текущий максимум и шаг. Java не формирует альтернативную вставку в lf_bids. Новая заявка владельца приостанавливает аукцион; заблокированный участник также приводит к приостановке связанных торгов.

Завершение отдельного лота вызывает lf_finalize_auction. Фоновое пакетное завершение вызывает lf_close_due_auctions. Приостановленные торги не закрываются автоматически. Без ставок вещь сохраняется в каталоге. Для победителя PL/pgSQL создаёт принятое дело с auction_id, диалог и передачу; вещь переходит через auctioned в reserved. Повтор завершения не создаёт вторую передачу. Продавец и победитель используют обычные два подтверждения; истёкшая подписка этому не препятствует. Односторонняя отмена результата торгов и отмена уже подтверждённой одной стороной

передачи запрещены; спор разбирает модератор. Оплата купленного лота не входит в предметную область курсовой.

18.5. Жалобы, администрирование и события

AdministrationService принимает жалобы, проверяет независимость модератора, блокирует пользователей и управляет служебными данными. Обычный модератор не может блокировать сотрудника без роли admin. Блокировка и изменение роли требуют причины. Администратор назначает и отзывает роли, создаёт и отключает тарифы, обновляет справочник организаций. Отзыв роли последнего действующего администратора запрещён; проверка выполняется под блокировкой записи роли и повторной проверкой полномочий инициатора. Аудит доступен только служебным ролям с ограниченным размером страницы. Для проверки профилей, жалоб, изменения ролей, тарифов и справочников сервисы дополняют триггерный аудит явными записями в той же транзакции.

OutboxService выбирает недоставленные события с FOR UPDATE SKIP LOCKED. Уведомления и delivered_at сохраняются в одной транзакции. Сообщение уведомляет другую сторону без копирования частного текста. Окончание последнего периода подписки создаёт одно устойчивое событие; активное продление исключает ложное уведомление. Решение по профилю и жалобе видно получателю в кабинете, в том числе до одобрения профиля. Событие активации подписки помечается доставленным без повторной вставки: исходное уведомление уже создаёт функция оплаты. SMTP-доставка с собственным повтором и внешняя рассылка ещё не подключены.

Таблица 20 — Соответствие прецедентов первого этапа реализации

Прецедент	Реализованные методы	Граница третьего этапа
UC-01 Получить доступ	register, confirmEmail, submitProfile, reviewProfile, authenticate, createOrder, payDemo, periods; восстановление доступа	Решение профиля и оплата в БД; HTTP и внешняя почта относятся к следующему этапу
UC-02 Опубликовать объявление	create, edit, attachImage, putPrivateAttribute, submit, moderate, archive	Автор готовит материал, независимый модератор публикует; двоичный адаптер — следующий этап

Прецедент	Реализованные методы	Граница третьего этапа
UC-03 Вернуть вещь	search, searchByState, detail, claim, send, messages, reserve, reject, cancel, confirm, organizations	Владелец и нашедший взаимодействуют через одно дело; хранитель-организация показывается как альтернатива
UC-04 Разрешить спор	complain, messages, send, reviewComplaint, resolve, block	Доступ сотрудника к диалогу только при открытой жалобе, независимое решение
UC-05 Продать вещь	requestPermission, reviewPermission, create, bid, finalizeAuction, closeDue, confirm	Допуск, конкурирующие ставки, итог PL/pgSQL и передача победителю
UC-06 Изменить полномочия	grantRole, revokeRole, audit	Актуальные права, обязательная причина и защита последнего администратора
UC-07 Актуализировать инструкцию	updateOrganization, organizations, audit	Администратор обновляет источник, участник получает справочную запись
UC-08 Изменить тариф	tariff, retireTariff, createOrder, payDemo, audit	Новый заказ получает новую цену; старые заказы и периоды сохраняют условия

19. Проверка и демонстрация на helios

19.1. Изолированные интеграционные проверки

Проверки выполняются с настоящим PostgreSQL на helios. Java-команда verify модуля tooling создаёт временный набор lf3check_* из SQL второго этапа в схеме s465826, заполняет его синтетическими данными и запускает Gradle test. Java подключается через локальный SSH-туннель; пароль базы передаётся в окружение дочернего процесса и не сохраняется в исходных файлах. Отдельная команда verify-db проверяет целостность и конкуренцию на наборе lfcheck_*.

Имена таблиц, функций, триггеров, индексов и последовательностей меняются согласованно. Проверочный набор не использует lf_* и объекты другой лабораторной. Перед запуском проверяется отсутствие lf3check_*. Если они уже существуют, скрипт останавливается и не удаляет их автоматически. После выполнения используются штатный безопасный drop.sql, проверка каталога объектов и сравнение значений исходных последовательностей.

ServiceIntegrationTest выполняется с откатом каждого сценария. TransactionIntegrationTest проверяет реальные коммиты, параллельные соединения и завершение аукциона по серверному времени. Все коммиты происходят внутри одноразового набора, который затем удаляется целиком. SqlNamesTest проверяет допустимые идентификаторы. Встроенная H2 и мок-реализация SQL-функций не используются.

Таблица 21 — Группы проверок третьего этапа

Группа	**Что проверяется**
Аккаунты	BCrypt, регистрация, одноразовый токен, модерация, восстановление и отзыв сессии
Подписки	Повтор заказа и события, продление, чужой заказ, откат ошибочной оплаты
Объявления	Публикация, новая модерация после правки, время, собственник, метаданные изображений
Приватность	Явная публичная карточка, закрытые признаки и доступ к диалогу
Возврат	Один резерв, отмена, два подтверждения, истечение подписки, решение модератора
Торги	Повтор ставки, смена содержимого ключа, продавец, заблокированный участник, приостановка, победитель
Администрирование	Полномочия, жалобы, тарифы, роли, уведомления и аудит
События	Доставка outbox и отсутствие дубликатов при повторе
Конкуренция	Два резервирования одной вещи и два подтверждения одного платежа
Изоляция	Сохранность исходных объектов, последовательностей и удаление проверочного набора

Актуальный общий прогон содержит 113 тестов, без ошибок и пропусков. Из них 57 проверяют хранение, предметные сервисы и идентификаторы SQL, а 56 — веб-уровень и файловый адаптер четвёртого этапа. Компиляция и bootJar завершены успешно. Подробные протоколы находятся в docs/part3/validation/integration.txt и isolation.txt; краткий результат по тестовым классам — в tests.json. Протокол запуска собранного приложения на helios сохранён в helios.txt, контрольная сумма установленного JAR — в deployment.txt.

Дополнительно повторно выполнена проверка базы второго этапа: 50 условий целостности, конкурирующие резервы и ставки, повтор оплаты, пакетное завершение и удаление только проверочного набора. Результат сохранён в database-regression.txt. Основные объекты курсовой и лабораторной сохранены.

19.2. Сборка и запуск

Локальная сборка требует JDK 17 и доступа к Maven Central. Gradle устанавливается через Wrapper; контрольная сумма дистрибутива задана в gradle-wrapper.properties.

```
cd D:\IS\course\backend
$env:JAVA_HOME='C:\Users\minec\.jdks\corretto-17.0.14'
.\gradlew.bat test bootJar
```

Без RUN_DB_TESTS интеграционные проверки пропускаются. Для полного прогона используется следующая Java-команда; SSH-пароль запрашивается скрытым вводом:

```
cd D:\IS\course
& "$env:JAVA_HOME\bin\java.exe" -Xmx256m `
-jar tooling/build/libs/course-tools.jar verify
```

На helios JAR расположен только в каталоге курсовой. Launcher использует DB_URL, DB_USER и DB_PASSWORD, а при отсутствии пароля читает подходящую запись .pgpass. Глобальные настройки пользователя и файлы другой лабораторной не меняются.

```
cd ~/poteryashki-course
java -Xmx256m -jar tooling/build/libs/course-tools.jar launch --demo
```

Для JVM заданы -Xms64m и -Xmx256m, поскольку автоматический выбор максимальной памяти на helios превышает доступный объём. Приложение запускается на установленном OpenJDK 21, а байткод собран для Java 17. Демонстрация читает синтетический аккаунт нашедшего, выполняет поиск функцией PL/pgSQL, показывает контакты метрополитена и число уведомлений.

На старте Hibernate проверяет соответствие 27 сущностей существующей схеме в режиме ddl-auto=validate. Приложение не создаёт и не удаляет таблицы, не использует общий flyway_schema_history лабораторной и

не запускает миграции другой работы. Скрипты DDL второго этапа остаются источником структуры базы.

19.3. Границы реализации

Третий этап завершает хранение и бизнес-логику. Веб-страницы, контроллеры Spring MVC, выпуск JWT, защищённые Cookie, HTTP-ограничение попыток входа, SMTP и двоичный MinIO-адаптер относятся к последующему этапу. Доступ к Java-сервисам предполагается из доверенного транспортного слоя: он должен получать пользователя из проверенной сессии, а не из произвольного параметра запроса. Отдельного публичного endpoint с передачей actorId нет.

Проверки третьего этапа охватывают доступ к базе, переходы состояний, транзакции, права участников и доставку внутренних уведомлений. Измерение производительности интерфейса и испытания резервного копирования, HTTPS и внешних интеграций в этот этап не входят.

20. Результаты третьего этапа

Разработаны диаграммы классов, 27 отображений таблиц, репозитории и семь предметных сервисов с общим контролем доступа. Критичные процессы используют функции и процедуру PL/pgSQL второго этапа. Работа Java-кода проверена на настоящем PostgreSQL, включая конкуренцию и откат ошибок. Собранный JAR запускается на helios в отдельном каталоге курсовой.

Источники третьего этапа

1. Пример структуры Java-проекта и отчёта: MyFilmList, фиксированный коммит.

2. Spring Boot 3.5: требования к среде.

3. Spring Framework: JdbcTemplate и обработка SQL.

4. Spring Framework: совместная работа JPA и JDBC.

5. PostgreSQL: блокировки и конкуренция.